

Chapter 7. Object Lifetime

One benefit of .NET's managed execution model is that the runtime can automate most of your application's memory management. I have shown numerous examples that create objects with the `new` keyword, and none has explicitly freed the memory consumed by these objects.

In most cases, you do not need to take any action to reclaim memory. The runtime provides a *garbage collector* (GC),¹ a mechanism that automatically discovers when objects are no longer in use and recovers the memory they had been occupying so that it can be used for new objects. However, there are certain usage patterns that can cause performance issues or even defeat the GC entirely, so it's useful to understand how it works. This is particularly important with long-running processes that could run for days (short-lived processes may be able to tolerate a few memory leaks).

The GC is designed to manage memory efficiently, but memory is not the only limited resource you may need to deal with. Some things have a small memory footprint in the CLR but represent something relatively expensive, such as a database connection or a handle from an OS API. The GC doesn't always deal with these effectively, so I'll explain `IDisposable`, the interface designed for dealing with things that need to be freed more urgently than memory.

Value types often have completely different rules governing their lifetime—some local variable values live only for as long as their containing method runs, for example. Nonetheless, value types sometimes end up acting like reference types and being managed by the GC. I will discuss why that can be useful, and I will explain the *boxing* mechanism that makes it possible.

Garbage Collection

The CLR maintains a *heap*, a service that provides memory for the objects and values whose lifetime is managed by the GC. Each time you construct an instance of a class with `new`, or you create a new array object, the CLR allocates a new heap block. The GC decides when to deallocate that block.

NOTE

If you are writing a .NET application that runs on an Android device using .NET's Xamarin tools, there will be two garbage collected heaps: one for .NET and one for Java. Normal C# activity in Xamarin applications uses the .NET heap, so Java's heap only enters the picture if you write C# code that uses Xamarin's services for manipulating Java objects. This is a .NET book, so I will be focusing on the .NET GC.

A heap block contains all the nonstatic fields for an object, or all the elements if it's an array. The CLR also adds a header, which is not directly visible to your program. This includes a pointer to a structure describing the object's type. This supports operations that depend on the real type of an object. For example, if you call `GetType` on a reference, the runtime uses this pointer to find out the type. (The type is often not completely determined by the static type of the reference, which could be an interface type or a base class of the actual type.) It's also used to work out which method to use when you invoke a virtual method or an interface member. The CLR also uses this to know how large the heap block is—the header does not include the block size, because the runtime can work that out from the object's type. (Most types are fixed size. There are only two exceptions, strings and arrays, which the CLR handles as special cases.) The header contains one other field, which is used for a variety of diverse purposes, including multithreaded synchronization and default hash code generation. Heap block headers are just an implementation detail, and different runtimes could choose different strategies.² However, it's useful to know what the overhead is. On a 32-bit system, the header is 8 bytes long, and if you're running in a 64-bit process, it takes 16 bytes. So an object that contained just one field of type `double` (an 8-byte type)

would consume 16 bytes in a 32-bit process, and 24 bytes in a 64-bit process.

Although objects (i.e., instances of a class) always live on the heap, instances of value types are different: some live on the heap, and some don't.³ The CLR stores some value-typed local variables on the stack, for example, but if the value is in an instance field of a class, the class instance will live on the heap, and that value will therefore live inside that object on the heap. And in some cases, a value will have an entire heap block to itself.

If you're using something through a reference type variable, then you are accessing something on the heap. It's important to clarify exactly what I mean by a reference type variable, because unfortunately, the terminology is a little confusing here: C# uses the term *reference* to describe two quite different things. For the purposes of this discussion, a reference is something you can store in a variable of a type that derives from `object` (but not from `ValueType`) or that is an interface type. This does not include every `in`-, `out`-, or `ref`-style method argument, nor `ref` variables or returns. Although those are references of a kind, a `ref int` argument is a reference to a value type, and that's not the same thing as a reference type. (The CLR actually uses a different term than C# for the mechanism that supports `ref`, `in`, and `out`: it calls these *managed pointers*, making it clear that they are rather different from object references.)

The managed execution model used by C# (and all .NET languages) means the CLR knows about every heap block your code creates, and also about every field, variable, and array element in which your program stores references. This information enables the runtime to determine at any time which objects are *reachable*—that is, those that the program could conceivably get access to in order to use its fields and other members. If an object is not reachable, then by definition the program will never be able to use it again. To illustrate how the CLR determines reachability, I've written a simple method, shown in [Example 7-1](#), that fetches web pages from my employer's website. (This is just meant to illustrate GC behavior, so it is slightly unrealistic: as explained in [“Optional Disposal”](#), you wouldn't normally create a new `HttpClient` for each request.)

Example 7-1. Using and discarding objects

```
public static string FetchUrl(string relativeUri)
{
    var baseUrl = new Uri("https://endjin.com/");
    var fullUri = new Uri(baseUrl, relativeUri);
    var w = new HttpClient();
    HttpResponseMessage response = w.Send(
        new HttpRequestMessage(HttpMethod.Get, fullUri));
    return new StreamReader(response.Content.ReadAsStream()).ReadToEnd();
}
```

The CLR analyzes the way in which we use local variables and method arguments. For example, although the `relativeUri` argument is in scope for the whole method, we use it just once as an argument when constructing the second `Uri` and then never use it again. A variable is described as *live* from the first point at which it receives a value up until the last point at which it is used. Method arguments are live from the start of the method until their final usage, unless they are unused, in which case they are never live. Local variables become live later; `baseUrl` becomes live once it has been assigned its initial value and then ceases to be live with its final usage, which in this example, happens at the same point as `relativeUri`. Liveness is an important property in determining whether a particular object is still in use.

To see the role that liveness plays, suppose that when [Example 7-1](#) reaches the line that constructs the `HttpClient`, the CLR doesn't have enough free memory to hold the new object. It could request more memory from the OS at this point, but it also has the option to try to free up memory from objects that are no longer in use, meaning that our program wouldn't need to consume more memory than it's already using.⁴ The next section describes the process that the CLR uses when it takes that second option.

Determining Reachability

.NET's basic approach is to determine which of the objects on the heap are reachable. If there's no way for a program to get hold of some object, it can safely be discarded. The CLR starts by determining all of the *root*

references in your program. A *root* is a storage location, such as a local variable, that could contain a reference and is known to have been initialized, and that your program could use at some point in the future without needing to go via some other object reference. Not all storage locations are considered to be roots. If an object contains an instance field of some reference type, that field is not a root, because before you can use it, you'd need to get hold of a reference to the containing object, and it's possible that the object itself is not reachable. However, a reference type static field is a root reference, because the program can read the value in that field at any time—the only situation in which that field will become inaccessible in the future is when the component that defines the type is unloaded, which in most cases will be when the program exits.

Local variables and method arguments are more interesting. Sometimes they are roots but sometimes not. It depends on exactly which part of the method is currently executing. A local variable or argument can be a root only if the flow of execution is currently inside the region in which that variable or argument is live. So, in [Example 7-1](#), `baseUrl` is a root reference only after it has had its initial value assigned and before the call to construct the second `Uri`, which is a rather narrow window. The `fullUri` variable is a root reference for slightly longer, because it becomes live after receiving its initial value and continues to be live during the construction of the `HttpClient` on the following line; its liveness ends only once `HttpRequestMessage` constructor has been called.

NOTE

When a variable's last use is as an argument in a method or constructor invocation, it ceases to be live when the method call begins. At that point, the method being called takes over—its own arguments are live at the start (except for arguments it does not use). However, they will typically cease to be live before the method returns. This means that in [Example 7-1](#), the object referred to by `fullUri` may cease to be accessible through root references before the `HttpRequestMessage` constructor returns.

Since the set of live variables changes as the program executes, the set of root references also evolves. To guarantee correct behavior in the face of

this moving target, the CLR can suspend all threads that are running managed code when necessary during garbage collection.

Live variables and static fields are not the only kinds of roots. Evaluation of expressions sometimes creates temporary objects, which need to stay alive for as long as necessary to complete the evaluation, so there can be some root references that don't correspond directly to any named entities in your code. And there are other types of root. For example, the `GCHandle` class lets you create new roots explicitly, which can be useful in interop scenarios to enable some unmanaged code to get access to a particular object. There are also situations in which roots are created implicitly. Certain kinds of applications can interoperate with non-.NET object-based systems (e.g., COM in Windows applications, or Java on Android), which can establish root references without explicit use of `GCHandle`—if the CLR needs to generate a wrapper making one of your .NET objects available to some other runtime, that wrapper will effectively be a root reference. Calls into unmanaged code may also involve passing pointers to memory on the heap, which will mean that the relevant heap block needs to be treated as reachable for the duration of the call. The broad principle is that roots will exist where necessary to ensure that objects that are still in use remain reachable.

Having built up a complete list of current root references for all threads, the GC works out which objects can be reached from these references. It looks at each reference in turn, and if non-null, the GC knows that the object it refers to is reachable. There may be duplicates—multiple roots may refer to the same object, so the GC keeps track of which objects it has already seen. For each newly discovered object, the GC adds all of the instance fields of reference type in that object to the list of references it needs to look at, again discarding duplicates. (This includes hidden fields generated by the compiler, such as those for automatic properties, which I described in [Chapter 3](#).) It does the same for each element of any reference-typed arrays it discovers. This means that if an object is reachable, so are all the objects to which it holds references. The GC repeats this process until it runs out of new references to examine. Any objects that it has *not* discovered to be reachable must be unreachable, because the GC is simply doing what the program does: a program can use only objects

that are accessible either directly or indirectly through its variables, temporary local storage, static fields, and other roots.

Going back to [Example 7-1](#), what would all this mean if the CLR decides to run the GC when we construct the `HttpClient`? The `fullUri` variable is still live, so the `Uri` it refers to is reachable, but the `baseUri` is no longer live. We did pass a copy of `baseUri` into the constructor for the second `Uri`, and if that had stored a copy of the reference in a field, then it wouldn't matter that `baseUri` is not live; as long as there's some way to get to an object by starting from a root reference, then the object is reachable. But as it happens, the second `Uri` won't do that, so the first `Uri` the example allocates would be deemed to be unreachable, and the CLR would be free to recover the memory it had been using.

One important upshot of how reachability is determined is that the GC is unfazed by circular references. This is one reason .NET uses GC instead of reference counting (another popular approach for automating memory management). If you have two objects that refer to each other, a reference counting scheme will consider both objects to be in use, because each is referred to at least once. But the objects may be unreachable—if there are no other references to the objects, the application will not have any way to use them. Reference counting fails to detect this, so it could cause memory leaks, but with the scheme used by the CLR's GC, the fact that they refer to each other is irrelevant—the GC will never get to either of them, so it will correctly determine that they are no longer in use.

Accidentally Defeating the Garbage Collector

Although the GC can discover ways that your program could reach an object, it has no way to prove that it necessarily will. Take the impressively idiotic piece of code in [Example 7-2](#). Although you'd never write code this bad, it makes a common mistake. It's a problem that usually crops up in more subtle ways, but I want to show it in a more obvious example first. Once I've shown how it prevents the GC from freeing objects that we're not going to be using, I'll describe a less straightforward but more realistic scenario in which this same problem often occurs.

Example 7-2. An appallingly inefficient piece of code

```
static void Main()
{
    var numbers = new List<string>();
    long total = 0;
    for (int i = 1; i < 100_000; ++i)
    {
        numbers.Add(i.ToString());
        total += i;
    }
    Console.WriteLine("Total: {total}, average: {total / numbers.Count}");
}
```

This adds together the numbers from 1 to 100,000 and then displays their average. The first mistake here is that we don't even need to do the addition in a loop, because there's a simple and very well-known closed-form solution for this sort of sum: $n*(n+1)/2$, with n being 100,000 in this case. That mathematical gaffe notwithstanding, this code does something even more stupid: it builds up a list containing every number it adds, but all it does with that list is retrieve its `Count` property to calculate an average at the end. Just to make things worse, the code converts each number into a string before putting it in the list. It never actually uses those strings. (I've shown the `Main` method declaration here to make it clear that `numbers` isn't used later on.) Obviously, this is a contrived example. Real examples of this kind of mistake tend to be better obfuscated. The purpose of this example is to show how you can run into a limitation of the GC.

Suppose the loop in [Example 7-2](#) has been running for a while—perhaps it's on its 90,000th iteration and is trying to add an entry to the `numbers` list. Suppose that the `List<string>` has used up its spare capacity, and the `Add` method will therefore need to allocate a new, larger internal array. The CLR may decide at this point to run the GC to see if it can free up some space. What will happen?

[Example 7-2](#) creates three kinds of objects: it constructs a `List<string>` at the start, it creates a new `string` each time around the loop by calling `ToString()` on an `int`, and more subtly, the `List<string>` will allocate

a `string[]` to hold references to those strings. Because we keep adding new items, it will have to allocate larger and larger arrays. (That array is an implementation detail of `List<string>`, so we can't see it directly.) So the question is: Which of these objects can the GC discard to make space for a larger array in the call to `Add`?

Our `numbers` variable remains live until the program's final statement, and we're looking at an earlier point in the code, so the `List<string>` object it refers to is reachable. The `string[]` array object it is currently using must also be reachable: it's allocating a newer, larger one, but it will need to copy the contents of the old one across to the new one, so the list must still have a reference to that current array stored in one of its fields. Since that array is still reachable, every string the array refers to will also be reachable. Our program has created 90,000 strings so far, and the GC will find all of them by starting at our `numbers` variable, looking at the fields of the `List<string>` object that refers to, and then looking at every element in the array that one of the list's private fields refers to.

The only allocated items that the GC might be able to collect are old `string[]` arrays that the `List<string>` created back when the list was smaller and that it no longer has a reference to. By the time we've added 90,000 items, the list will probably have resized itself quite a few times. So depending on when the GC last ran, it will probably be able to find a few of these now-unused arrays. But more interesting here is what it cannot free.

The program will never use any of the 90,000 strings it has created, so ideally, we'd like the GC to free up the memory they occupy—they will be taking up a few megabytes. We can see very easily that these strings are not used, because this is such a short program. But the GC will not know that; it bases its decisions on reachability, and it correctly determines that all 90,000 strings are reachable by starting at the `numbers` variable. And as far as the GC is concerned, it's entirely possible that the list's `Count` property, which we use after the loop finishes, will look at the contents of the list.

You and I happen to know that it won't, because it doesn't need to, but that's because we know what the `Count` property means. For the GC to

infer that our program will never use any of the list's elements directly or indirectly, it would need to know what `List<string>` does inside its `Add` and `Count` methods. This would mean analysis with a level of detail far beyond the mechanisms I've described, which could make GCs considerably more expensive. Moreover, even with the serious step up in complexity required to detect which reachable objects this example will never use, in more realistic scenarios the GC is unlikely to be able to make predictions that were significantly better than relying on reachability alone.

For example, a much more plausible way to run into this problem is in a cache. If you write a class that caches data that is expensive to fetch or calculate, imagine what would happen if your code only ever added items to the cache and never removed them. All of the cached data would be reachable for as long as the cache object itself is reachable. The problem is that your cache will consume more and more space, and unless your computer has sufficient memory to hold every piece of data that your program could conceivably need to use, it will eventually run out of memory.

A naive developer might complain that this is supposed to be the GC's problem. The whole point of GC is meant to be that I don't need to think about memory management, so why am I running out of memory all of a sudden? But, of course, the problem is that the GC has no way of knowing which objects are safe to remove.

Not being clairvoyant, it cannot accurately predict which cached items your program may need in the future—if the code is running in a server, future cache usage could depend on what requests the server receives, something the GC cannot predict. So although it's possible to imagine memory management smart enough to analyze something as simple as [Example 7-2](#), in general, this is not a problem the GC can solve. Thus, if you add objects to collections and keep those collections reachable, the GC will treat everything in those collections as being reachable. It's your job to decide when to remove items.

Collections are not the only mechanism where certain usage patterns can mislead the GC. As I'll show in [Chapter 9](#), there's a common scenario in

which careless use of events can cause memory leaks. More generally, if your program makes it possible for an object to be reached, the GC has no way of working out whether you're going to use that object again, so it has to be conservative.

That said, there is a technique for mitigating this with a little help from the GC.

Weak References

Although the GC will follow ordinary references in a reachable object's fields, it is possible to hold a *weak reference*. The GC does not follow weak references, so if the only way to reach an object is through weak references, the GC behaves as though the object is not reachable and will remove it. A weak reference provides a way of telling the CLR, "Do not keep this object around on my account, but for as long as something else needs it, I would like to be able to get access to it." [Example 7-3](#) shows a cache that uses `WeakReference<T>`.

Example 7-3. Using weak references in a cache

```
public class WeakCache<TKey, TValue>
    where TKey : notnull
    where TValue : class
{
    private readonly Dictionary<TKey, WeakReference<TValue>> _cache = new();

    public void Add(TKey key, TValue value)
    {
        _cache.Add(key, new WeakReference<TValue>(value));
    }

    public bool TryGetValue(
        TKey key, [NotNullWhen(true)] out TValue? cachedItem)
    {
        if (_cache.TryGetValue(key, out WeakReference<TValue>? entry))
        {
            bool isAlive = entry.TryGetTarget(out cachedItem);
            if (!isAlive)
            {
                _cache.Remove(key);
            }
        }
    }
}
```

```

        }
        return isAlive;
    }
    else
    {
        cachedItem = null;
        return false;
    }
}
}

```

This cache stores all values via a `WeakReference<T>`. Its `Add` method passes the object to which we'd like a weak reference as the constructor argument for a new `WeakReference<T>`. The `TryGetValue` method attempts to retrieve a value previously stored with `Add`. It first checks to see if the dictionary contains a relevant entry. If it does, that entry's value will be the `WeakReference<T>` we created earlier. My code calls that weak reference's `TryGetTarget` method, which will return `true` if the object is still available and `false` if it has been collected.

NOTE

Availability doesn't necessarily imply reachability. The object may have become unreachable since the most recent GC. Or there may not even have been a GC since the object was allocated. `TryGetTarget` can tell you only whether the GC has detected that it is eligible for collection.

If the object is available, `TryGetTarget` provides it through an `out` parameter, and this will be a strong reference. So, if this method returns `true`, we don't need to worry about any race condition in which the object becomes unreachable moments later—the fact that we've now stored that reference in the variable the caller supplied via the `cachedItem` argument will keep the target alive. If `TryGetTarget` returns `false`, my code removes the relevant entry from the dictionary, because it represents an object that no longer exists. That's important because although a weak reference won't keep its target alive, the `WeakReference<T>` is an object in its own right, and the GC can't free it until I've removed it from this dictionary. (In real application code you would also want to scan the whole dictionary periodically to remove all such entries instead of just

checking individual entries only when something asks for them.)

[Example 7-4](#) tries this code out, forcing a couple of garbage collections so we can see it in action. (This splits each stage into separate methods with inlining disabled because otherwise, .NET's JIT compiler will inline these methods, and it ends up creating hidden temporary variables that can cause the array to remain reachable longer than it should, distorting the results of this test.)

Example 7-4. Exercising the weak cache

```
internal class Program
{
    private static readonly WeakCache<string, byte[]> cache = new();
    private static byte[]? data = new byte[100];

    private static void Main(string[] args)
    {
        AddData();
        CheckStillAvailable();

        GC.Collect();
        CheckStillAvailable();

        SetOnlyRootToNull();
        GC.Collect();
        CheckNoLongerAvailable();
    }

    [MethodImpl(MethodImplOptions.NoInlining)]
    private static void AddData()
    {
        cache.Add("d", data!);
    }

    [MethodImpl(MethodImplOptions.NoInlining)]
    private static void CheckStillAvailable()
    {
        Console.WriteLine("Retrieval: " +
            cache.TryGetValue("d", out byte[]? fromCache));
        Console.WriteLine("Same ref? " +
            object.ReferenceEquals(data, fromCache));
    }
}
```

```

[MethodImpl(MethodImplOptions.NoInlining)]
private static void SetOnlyRootToNull()
{
    data = null;
}

[MethodImpl(MethodImplOptions.NoInlining)]
private static void CheckNoLongerAvailable()
{
    byte[]? fromCache;
    Console.WriteLine("Retrieval: " + cache.TryGetValue("d", out fromCache));
    Console.WriteLine("Null? " + (fromCache == null));
}
}

```

This begins by adding a reference to a 100-byte array to the cache. It also stores a reference to the same array in a static field called `data`, keeping the array reachable until the code calls `SetOnlyRootToNull`, which sets `data` to `null`. The example tries to retrieve the value from the cache immediately after adding it and also uses `object.ReferenceEquals` just to check that the value we get back really refers to the same object that we put in. Then I force a garbage collection and try again. (This sort of artificial test code is one of the few situations in which you'd want to do this—see the section [“Forcing Garbage Collections”](#).) Since the `data` field still holds a reference to the array, the array is still reachable, so we would expect the value still to be available from the cache. Next I set `data` to `null`, so my code is no longer keeping that array reachable. The only remaining reference is a weak one, so when I force another GC, we expect the array to be collected and the final lookup in the cache to fail. To verify this, I check both the return value, expecting `false`, and the value returned through the `out` parameter, which should be `null`. And that is exactly what happens when I run the program, as you can see:

```

Retrieval: True
Same ref?  True
Retrieval: True
Same ref?  True
Retrieval: False
Null?     True

```

NOTE

Writing code to illustrate GC behavior means entering treacherous territory. The principles of operation remain the same, but the exact behavior of small examples changes over time, often due to optimizations performed during JIT compilation. It's entirely possible that if you try these examples, you might see different behavior due to changes in the runtime since going to press.

Later, I will describe finalization, which complicates matters by introducing a twilight zone in which the object has been determined to be unreachable but has not yet gone. Objects that are in this state are typically of little use, so by default, a weak reference will treat objects waiting for finalization as though they have already gone. This is called a *short weak reference*. If, for some reason, you need to know whether an object has really gone (rather than merely being on its way out), the

`WeakReference<T>` class's constructor has overloads, some of which can create a *long weak reference*, which provides access to the object even in this zone between unreachability and final removal.

Reclaiming Memory

So far, I've described how the CLR determines which objects are no longer in use but not what happens next. Having identified the garbage, the runtime must then collect it. The CLR uses different strategies for small and large objects. (By default, the .NET CLR defines a large object as one bigger than 85,000 bytes. Mono sets the bar lower at 8,000 bytes.) Most allocations involve small objects, so I'll write about those first.

The CLR tries to keep the heap's free space contiguous. That's easy when the application first starts up, because there's nothing but free space, and it can keep things contiguous by allocating memory for each new object directly after the last one. But after the first GC occurs, the heap is unlikely to look so neat. Most objects have short lifetimes, and it's common for the majority of objects allocated after any one GC to be unreachable by the time the next GC runs. However, some will still be in use. From time to time, applications create objects that hang around for longer, and whatever work was in progress when the GC ran will probably be using some objects, so the most recently allocated heap blocks are likely still to

be in use. This means that the end of the heap might look something like [Figure 7-1](#), where the shaded rectangles are the reachable blocks, and the white ones show blocks that are no longer in use.

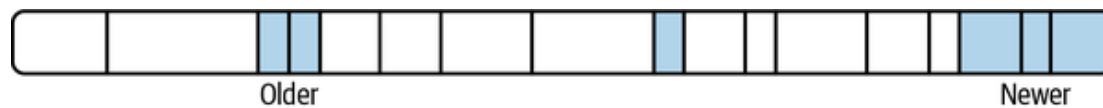


Figure 7-1. Section of heap with some reachable objects

One possible allocation strategy would be to start using these empty blocks as new memory is required, but there are a couple of problems with that approach. First, it tends to be wasteful, because the blocks the application requires will probably not fit precisely into the holes available. Second, finding a suitable empty block can be somewhat expensive, particularly if there are lots of gaps and you're trying to pick one that will minimize waste. It's not impossibly expensive, of course—lots of heaps work this way—but it's a lot costlier than the initial situation where each new block could be allocated directly after the last one because all the spare space was contiguous. The expense of heap fragmentation is non-trivial, so the CLR typically tries to get the heap back into a state where the free space is contiguous. As [Figure 7-2](#) shows, it moves all the reachable objects toward the start of the heap so that all the free space is at the end, which puts it back in the favorable situation of being able to allocate new heap blocks one after another in the contiguous lump of free space.

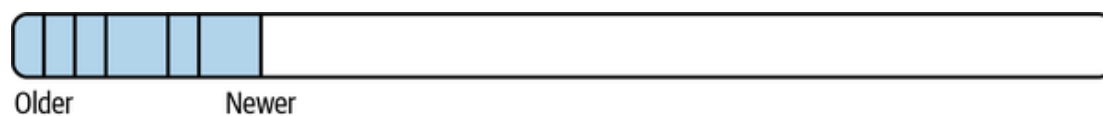


Figure 7-2. Section of heap after compaction

The runtime has to ensure that references to these relocated blocks continue to work after the blocks have moved. The CLR happens to implement references as pointers (although nothing requires this—a reference is just a value that identifies some particular instance on the heap). It already knows where all the references to any particular block are because it had to find them to discover which blocks were reachable. It adjusts all these pointers when it moves the block.

Besides making heap block allocation a relatively cheap operation, compaction offers another performance benefit. Because blocks are allocated into a contiguous area of free space, objects that were created in quick succession will typically end up right next to each other in the heap. This is significant, because the caches in modern CPUs tend to favor locality (i.e., they perform best when related pieces of data are stored close together).

The low cost of allocation and the high likelihood of good locality can sometimes mean that garbage-collected heaps offer better performance than traditional heaps that require the program to free memory explicitly. This may seem surprising, given that the GC appears to do a lot of extra work that is unnecessary in a noncollecting heap. Some of that “extra” work is nothing of the sort, however—something has to keep track of which objects are in use, and traditional heaps just push that housekeeping overhead into our code. However, relocating existing memory blocks comes at a price, so the CLR uses some tricks to minimize the amount of copying it needs to do.

The older an object is, the more expensive it will be for the CLR to compact the heap once it finally becomes unreachable. If the most recently allocated object is unreachable when the GC runs, compaction is free for that object: there are no more objects after it, so nothing needs to be moved. Compare that with the first object your program allocates—if that becomes unreachable, compaction would mean moving every reachable object on the heap. More generally, the older an object is, the more objects will be put after it, so the more data will need to be moved to compact the heap. Copying 20 MB of data to save 20 bytes does not sound like a great trade-off. So the CLR will often defer compaction for older parts of the heap.

To decide what counts as “old,” the .NET runtime divides the heap into *generations*.⁵ The boundaries between generations move around at each GC, because generations are defined in terms of how many GCs an object has survived. Any object allocated after the most recent GC is in generation 0, because it has not yet survived any collections. When the GC next runs, generation 0 objects that are still reachable will be moved as necessary to compact the heap and will then be deemed to be in generation 1.

Objects in generation 1 are not yet considered to be old. A GC will typically occur while the code is right in the middle of doing things—after all, it runs when space on the heap is being used up, and that won't happen if the program is idle. So there's a high chance that some of the recently allocated objects represent work in progress, and although they are currently reachable, they will become unreachable shortly. Generation 1 acts as a sort of holding zone while we wait to see which objects are short-lived and which are longer-lived.

As the program continues to execute, the GC will run from time to time, promoting new, surviving objects into generation 1. Some of the objects in generation 1 will become unreachable. However, the GC does not necessarily compact this part of the heap immediately—it may allow a few generation 0 collections and compactions in between each generation 1 compaction, but it will happen eventually. Objects that survive this stage are moved into generation 2, which is the oldest generation.

The CLR attempts to recover memory from generation 2 much less frequently than from other generations. Research shows that in most applications, objects that survive into generation 2 are likely to remain reachable for a long time, so when one of those objects does eventually become unreachable, it's likely to be very old, as will be the objects around it. This means that compacting this part of the heap to recover the memory is costly for two reasons: not only will this old object probably be followed by a large number of other objects (requiring a large volume of data to be copied), but also the memory it occupied might not have been used for a long time, meaning it's probably no longer in the CPU's cache, slowing down the copy even further. And the caching costs will continue after collection, because if the CPU has had to shift megabytes of data around in old areas of the heap, this will probably have the side effect of flushing other data out the CPU's cache. Cache sizes can be as small as 512 KB at the low-power, low-cost end of the spectrum, and can be over 90 MB in high-end, server-oriented chips, but in the midrange, anything from 2 MB to 16 MB of cache is typical, and many .NET applications' heaps will be larger than that. Most of the data the application had been using would have been in the cache right up until the generation 2 GC but would be gone once the GC has finished. So when the GC completes and normal ex-

ecution resumes, the code will run in slow motion for a while until the data the application needs is loaded back into the cache.

Generations 0 and 1 are sometimes referred to as the *ephemeral* generations, because they mostly contain objects that exist only for a short while. (The part of Mono's heap that serves a similar purpose is called the *nursery*, because it's for young objects.) The contents of these parts of the heap will often be in the CPU's cache because they will have been accessed recently, so compaction is not particularly expensive for these sections. Moreover, because most objects have a short lifetime, the majority of memory that the GC is able to collect will be from objects in these first two generations, so these are likely to offer the greatest reward (in terms of memory recovered) in exchange for the CPU time expended. So it's common to see several ephemeral collections per second in a busy program, but it's also common for several minutes to elapse between successive generation 2 collections.

The CLR has another trick up its sleeve for generation 2 objects. They often don't change much, so there's a high likelihood that during the first phase of a GC—in which the runtime detects which objects are reachable—it would be repeating some work it did earlier, because it will follow exactly the same references and produce the same results for significant subsections of the heap. The CLR employs mechanisms to detect when older heap blocks are modified. This enables it to rely on summarized results from earlier GC operations instead of having to redo all of the work every time.

How does the GC decide whether to collect just from generation 0 or also from 1 or even 2? Collections for all three generations are triggered by using up a certain amount of memory. So, for generation 0 allocations, once you have allocated some particular number of bytes since the last GC, a new GC will occur. The objects that survive this will move into generation 1, and the CLR keeps track of the number of bytes added to generation 1 since the last generation 1 collection; if that number exceeds a threshold, generation 1 will be collected too. Generation 2 works in the same way. The thresholds are not documented, and in fact they're not even constant; the CLR monitors your allocation patterns and modifies these thresholds to try to find a good balance for making efficient use of memory, minimiz-

ing the CPU time spent in the GC and avoiding the excessive latency that could arise if the CLR waited a very long time between collections, leaving huge amounts of work to do when the collection finally occurs.

NOTE

This explains why, as mentioned earlier, the CLR doesn't necessarily wait until it has actually run out of memory before triggering a GC. It may be more efficient to run one sooner.

You may be wondering how much of the preceding information is of practical significance. After all, the bottom line would appear to be that the CLR ensures that heap blocks are kept around for as long as they are reachable, and that sometime after they become unreachable, it will eventually reclaim their memory, and it employs a strategy designed to do this efficiently. Are the details of this generational optimization scheme relevant to a developer? They are insofar as they tell us that some coding practices are likely to be more efficient than others.

The most obvious upshot of the process is that the more objects you allocate, the harder the GC will have to work. But you'd probably guess that without knowing anything about the implementation. More subtly, larger objects cause the GC to work harder—collections for each generation are triggered by the amount of memory your application uses. So bigger objects don't just increase memory pressure, they also end up consuming more CPU cycles as a result of triggering more frequent GCs.

Perhaps the most important fact to emerge from an understanding of the generational nature of the collector is that the length of an object's lifetime has an impact on how hard the GC must work. Objects that live for a very short time are handled efficiently, because the memory they use will be recovered quickly in a generation 0 or 1 collection, and the amount of data that needs to be moved to compact the heap will be small. Objects that live for an extremely long time are also OK, because they will end up in generation 2. They will not be moved about often, because collections are infrequent for that part of the heap. However, although very short-lived and very long-lived objects are handled efficiently, objects that live

long enough to get into generation 2 but not much longer are a problem. Microsoft occasionally describes this occurrence as a *midlife crisis*.

If your application regularly creates lots of objects making it into generation 2 that go on to become unreachable, the CLR will need to perform collections on generation 2 more often than it otherwise might. (In fact, generation 2 is collected only during a *full collection*, which also collects free space previously used by large objects.) These are usually significantly more expensive than other collections. Compaction requires more work with older objects, but also, more housekeeping is required when disrupting the generation 2 heap. The picture the CLR has built up about reachability within this section of the heap may need to be rebuilt, which incurs a cost. There's a good chance that most of this part of the heap will not be in the CPU's cache either, so working with it can be slow.

Full GCs consume significantly more CPU time than collections in the ephemeral generations. In UI applications, this can cause delays long enough to be irritating for the user, particularly if parts of the heap had been paged out by the OS. In server applications, full collections may cause significant blips in the typical time taken to service a request. Such problems are not the end of the world, and as I'll describe later, the CLR offers some mechanisms to mitigate these kinds of issues. Even so, minimizing the number of objects that survive to generation 2 is good for performance. You would need to consider this when designing code that caches interesting data in memory—a cache aging policy that failed to take the GC's behavior into account could easily behave inefficiently, and if you didn't know about the perils of middle-aged objects, it would be hard to work out why. Also, as I'll show later in this chapter, the midlife crisis issue is one reason you might want to avoid C# destructors where possible.

I have left out some heap operational details, by the way. For example, I've not talked about how the GC typically dedicates sections of the address space to the heap in fixed-size chunks, nor the details of how it commits and releases memory. Interesting though these mechanisms are, they have much less relevance to how you design your code than an awareness of the assumptions that a generational GC makes about typical object lifetimes. The details also tend to change—recent releases of .NET

have made significant modifications to the details of GC operation to improve performance, but the basic principles have remained the same.

There's one last thing to talk about on the topic of collecting memory from unreachable objects. As mentioned earlier, large objects work differently. There's a separate heap called, appropriately enough, the *large object heap* (LOH), and the .NET runtime uses this for any object larger than 85,000 bytes;⁶ Mono's runtime uses an 8,000-byte threshold, because it is often used in more memory-constrained environments. That's just the object itself, not the sum total of all the memory an object allocates during construction. An instance of the `GreedyObject` class in [Example 7-5](#) would be tiny—it needs only enough space for a single reference, plus the heap block overhead. In a 32-bit process, that would be 4 bytes for the reference and 8 bytes of overhead, and in a 64-bit process, it would be twice as large. However, the array to which it refers is 400,000 bytes long, so that would go on the LOH, while the `GreedyObject` itself would go on the ordinary heap.

Example 7-5. A small object with a large array

```
public class GreedyObject
{
    public int[] MyData = new int[100_000];
}
```

It's technically possible to create a class whose instances are large enough to require the LOH, but it's unlikely to happen outside of generated code or highly contrived examples. In practice, most LOH heap blocks will contain arrays and possibly strings.

The biggest difference between the LOH and the ordinary heap is that the GC does not usually compact the LOH, because copying large objects is expensive. (Applications can request that the LOH be compacted at the next full GC. But applications that do not explicitly request this will never have their LOH compacted in current CLR implementations.) It works more like a traditional C heap: the CLR maintains a list of free blocks and decides which block to use based on the size requested. However, the list of

free blocks is populated by the same unreachability mechanism as is used by the rest of the heap.

Lightening the Load with Inline Arrays

The more objects we create, the more work the GC needs to do. C# 12.0 adds a new mechanism that can help us to allocate fewer objects. If we have written a type that always has an associated array (e.g., `List<T>` uses an array internally), that normally means that each instance of that type requires two allocations on the heap, one for itself and another for its array. However, in cases where the array always has the same number of elements, C# 12.0 gives us the option to define that as an *inline array*, meaning that the array elements can live inside another type's memory (just like a value-typed field) instead of requiring their own heap block.

WARNING

Inline arrays are intended for highly performance-sensitive scenarios. They add some complexity, they do not work on .NET Framework, and they are not as flexible as normal arrays. You should use them only in scenarios where performance profiling demonstrates that they make a useful difference.

To enable this capability without adding a whole new feature to the .NET type system, inline arrays are essentially just a particular kind of `struct`. It has always been possible to write code such as [Example 7-6](#). Historically, performance-sensitive libraries have often used exactly this sort of type to avoid allocating small fixed-size arrays.

Example 7-6. Emulating fixed-size arrays before C# 12.0

```
public struct ThreeIntegersPseudoArray
{
    public int Element0;
    public int Element1;
    public int Element2;
}
```

The main problem with that approach is that it is cumbersome. We need to define a field for each “array” element. We can’t use the normal array indexer syntax unless we define a custom indexer, and since we need to write a different type for each array size, we most likely won’t want to write custom indexers for all of them.

C# 12.0’s new inline array feature provides a better way to write this kind of type. The `ThreeIntegers` type shown in [Example 7-7](#) serves the same purpose as the one in [Example 7-6](#): because it is annotated with the `InlineArray(3)` attribute it will contain exactly three `int` values, and since it is a `struct`, it does not require its own heap block. I haven’t had to declare all three elements explicitly. I have defined a single field, but in an inline array type this field doesn’t exist at runtime—it is only there to indicate the element type. And this type automatically supports array indexer syntax.

Example 7-7. A fixed-size inline array type

```
[System.Runtime.CompilerServices.InlineArray(3)]
public struct ThreeIntegers
{
    private int _element0;
}
```

[Example 7-8](#) declares a local variable of type `ThreeIntegers`. Since this is a value type, we don’t need to use `new`—the `default` keyword here initializes all elements to zero. It won’t need its own heap block. Where possible, the compiler will store these values on the stack as with any other value-typed local variables. And in cases where it can’t do that (e.g., iterators or `async` methods), this variable would live inside the same heap-allocated type that holds all the other local variables, so we can use it without causing more allocations than would have happened in any case. If I declare a field of type `ThreeIntegers`, then just as with any other value type, its elements will live inside the containing type.

Example 7-8. Using a fixed-size inline array type

```
ThreeIntegers t = default;
t[0] += 1;
```

```
Console.WriteLine(t[0]);  
Console.WriteLine(t[1] + t[2]);
```

It may seem odd to have to define a type for each different array size. You might be wondering why we don't apply the `InlineArray` attribute to a field instead. The downside with that approach is that it would have been a more disruptive change. It would either have required a change to the long-established fact that value types always have a fixed size, or it would have meant not using value types at all for this feature, and introducing some new way of embedding a value inside another type. So although it is a little cumbersome to have to define a distinct type for each array size, the big advantage is that it's a relatively small change to the type system—it's really just an easier way to do what people had already been doing for years with code like [Example 7-6](#).

Garbage Collector Modes

Although the .NET runtime will tune some aspects of the GC's behavior at runtime (e.g., by dynamically adjusting the thresholds that trigger collections for each generation), it also offers a configurable choice between various modes designed to suit different kinds of applications. These fall into two broad categories—workstation and server, and then in each of these you can either use background or nonconcurrent collections. Background collection is on by default, but the default top-level mode depends on the project type: for console applications and applications using a GUI framework such as WPF, the GC runs in workstation mode, but ASP.NET Core web applications change this to server mode. You can control the GC mode explicitly by defining a `ServerGarbageCollection` property in your `.csproj` file, as [Example 7-9](#) shows. This can go anywhere inside the root `Project` element.

Example 7-9. Enabling server GC in a project file

```
<PropertyGroup>  
  <ServerGarbageCollection>true</ServerGarbageCollection>  
</PropertyGroup>
```

NOTE

This property makes the build system add a setting to the *YourApplication.runtimeconfig.json* file that it generates for your application. This contains a `configProperties` section, which can contain one or more *CLR host configuration knobs*. Enabling server GC in the project file sets the `System.GC.Server` knob to `true` in this configuration file. All GC settings are also controlled through configuration knobs, as are some other CLR behaviors, such as the JIT compiler mode.

The workstation modes are designed for the workloads that client-side code typically has to deal with, in which the process is usually working on either a single task or a small number of tasks at any one time. Workstation mode offers two variations: nonconcurrent and background.

In background mode (the default), the GC minimizes the amount of time for which it suspends threads during a GC. There are certain phases of the GC in which the CLR has to suspend execution to ensure consistency. For collections from the ephemeral generations, threads will be suspended for the majority of the operation. This is usually fine because these collections normally run very quickly. Full collections are the problem, and it's these that the background mode handles differently. Not all of the work done in a collection really needs to bring everything to a halt, and background mode exploits this, enabling full (generation 2) collections to proceed on a background thread without forcing other threads to block until that collection completes. This can enable machines with multiple processor cores (most machines, these days) to perform full GC collections on one core while other cores continue with productive work. It is especially useful in applications with a UI, because it reduces the likelihood of an application becoming unresponsive due to GCs.

The nonconcurrent mode is designed to optimize throughput on a single processor with a single core. It can be more efficient, because background GC uses slightly more memory and more CPU cycles for any particular workload than nonconcurrent GC in exchange for the lower latency. For some workloads, you may find your code runs faster if you set the `ConcurrentGarbageCollection` property to `false` in your project file. For most client-side code, the greatest concern is to avoid delays that are

long enough to be visible to users. Users are more sensitive to unresponsiveness than they are to suboptimal average CPU utilization, so for interactive applications, using slightly more memory and CPU cycles in exchange for improved perceived performance is usually a good trade-off.

Server mode is significantly different than workstation mode. It is available only when you have multiple hardware threads; e.g., a multicore CPU or multiple physical CPUs. (If you have enabled server GC but your code ends up running on a single-core machine,⁷ it falls back to using the workstation GC.) Its availability has nothing to do with which OS you're running, by the way—for example, server mode is available on nonsERVER and server editions of Windows alike if you have suitable hardware, and workstation mode is always available. Server mode is able to give each processor core its own section of the heap, so when a thread is working on its own problem independently of the rest of the process, it can allocate heap blocks with minimal contention. In server mode, the CLR creates several threads dedicated to GC, one for each logical CPU in the machine. These run with higher priority than normal threads, so when GCs do occur, all available CPU cores go to work on their own heaps, which can provide better throughput with large heaps than workstation mode.

NOTE

Objects created by one thread can still be accessed by others—logically, the heap is still a unified service. Server mode is just an implementation strategy optimized for workloads where each thread works on its own jobs mostly in isolation. Be aware that it works best if the jobs all have similar heap allocation patterns.

Until recently, these characteristics of server mode that enable it to make full use of a machine's resources could cause problems for some deployment models. If you have a server that does just one job, implemented as a single .NET process, you will want the available resources to be dedicated to that one process, so in these cases it's good that the GC tries to use all CPU cores simultaneously during collections, and that it has historically tended to be more eager to fully exploit the available memory than workstation mode. However, if a single server hosts a mix of workloads across multiple processes, you don't want them all acting that way, be-

cause contention for resources could reduce efficiency. .NET 8.0 made significant improvements in this area. It introduced a server GC feature called Dynamic Adaptation to Application Sizes (DATAS), in which server GC still makes full use of available resources if the application has high demands, but is more frugal when the usage doesn't justify this. This adaptation means that if a process has a highly variable workload, it can make full use of available resources during bursts of high activity, but when the load reduces it will relinquish these resources much sooner than in earlier .NET versions. It's now common practice to use container systems such as Docker to handle a mix of workloads efficiently on shared hardware, and this improved adaptability means server GC can now work better in those scenarios.

DATAS is off by default. You can enable it by adding `<GarbageCollectionAdaptationMode>1</GarbageCollectionAdaptationMode>` in a `PropertyGroup` in your `.csproj` file. This adds a `System.GC.DynamicAdaptationMode` property with a value of `1` to the `configProperties` section of your `.runtimeconfig.json` file.

Another feature of server GC is that it favors throughput over response time. In particular, collections happen less frequently, because this tends to increase the throughput benefits that multi-CPU collections can offer, but it also means that each individual collection takes longer.

As with workstation GC, the server GC uses background collection by default. In some cases, you may find you can improve throughput by disabling it, but be wary of the problems this can cause. The duration of a full collection in nonconcurrent server mode can cause serious delays in responsiveness on a website, for example, especially if the heap is large. You can mitigate this in a couple of ways. You can request notifications shortly before the collection occurs (using the `System.GC` class's `RegisterForFullGCNotification`, `WaitForFullGCApproach`, and `WaitForFullGCComplete` methods), and if you have a server farm, a server that's running a full GC may be able to ask the load balancer to avoid passing it requests until the GC completes. The simpler alternative is to leave background collection enabled. Since background collections allow application threads to continue to run and even to perform generation 0 and 1 collections while the full collection proceeds in the back-

ground, it significantly improves the application's response time during collections while still delivering the throughput benefits of server mode.

Temporarily Suspending Garbage Collections

It is possible to ask .NET to disallow GC while a particular section of code runs. This is useful if you are performing time-sensitive work. Windows, macOS, and Linux are not real-time operating systems, so there are never any guarantees, but temporarily ruling out GCs at critical moments can nonetheless be useful for reducing the chances of things going slowly at the worst possible moment. Be aware that this mechanism works by bringing forward any GC work that might otherwise have happened in the relevant section of code, so this can cause GC-related delays to happen earlier than they otherwise would have. It only guarantees that once your designated region of code starts to run, there will be no further GCs if you meet certain requirements—in effect, it gets necessary delays out of the way before the time-sensitive work begins.

The `GC` class offers a `TryStartNoGCRegion` method, which you call to indicate that you want to begin some work that needs to be free from GC-related interruption. You must pass in a value indicating how much memory you will need during this work, and it will attempt to ensure that at least that much memory is available before proceeding (performing a GC to free up that space if necessary). If the method indicates success, then as long as you do not consume more memory than requested, your code will not be interrupted by the GC. You call `EndNoGCRegion` once you have finished the time-critical work, enabling the GC to return to its normal operation. If, before it calls `EndNoGCRegion`, your code uses more memory than you requested, the CLR may have to perform a GC, but it will only do so if it absolutely cannot avoid it until you call `EndNoGCRegion`.

Although the single-argument form of `TryStartNoGCRegion` will perform a full GC if necessary to meet your request, some overloads take a `bool`, enabling you to tell it that if a full blocking GC will be required to free up the necessary space, you'd prefer to abort. There are also overloads in which you can specify your memory requirements on the ordinary heap and the large object heap separately.

Accidentally Defeating Compaction

Heap compaction is an important feature of the CLR's GC, because it has a strong positive impact on performance. Certain operations can prevent compaction, and that's something you'll want to minimize, because fragmentation can increase memory use and reduce performance significantly.

To be able to compact the heap, the CLR needs to be able to move heap blocks around. Normally, it can do this because it knows all of the places in which your application refers to heap blocks, and it can adjust all the references when it relocates a block. But what if you're calling an OS API that works directly with the memory you provide? For example, if you read data from a file or a network socket, how will that interact with GC?

If you use system calls that read or write data using devices such as the hard drive or network interface, these normally work directly with your application's memory. If you read data from the disk, the OS may instruct the disk controller to put the bytes directly into the memory your application passed to the API. The OS will perform the necessary calculations to translate the virtual address into a physical address. (With virtual memory, the value your application puts in a pointer is only indirectly related to the actual address in your computer's RAM.) The OS will lock the pages into place for the duration of the I/O request to ensure that the physical address remains valid. It will then supply the disk system with that address. This enables the disk controller to copy data from the disk directly into memory, without needing further involvement from the CPU. This is very efficient but runs into problems when it encounters a compacting heap. What if the block of memory is a `byte[]` array on the heap? Suppose a GC occurs between us asking to read the data and the disk being able to supply the data. (The chances are fairly high; a mechanical disk with spinning platters can take 10 ms or more to start supplying data, which is an age in CPU terms.) If the GC decided to relocate our `byte[]` array to compact the heap, the physical memory address that the OS gave the disk controller would be out of date, so when the controller started putting data into memory, it would be writing to the wrong place.

There are three ways the CLR could deal with this. One would be to make the GC wait—heap relocations could be suspended while I/O operations are in progress. But that’s a nonstarter in many scenarios; a busy server can run for days without ever entering a state in which no I/O operations are in progress. In fact, the server doesn’t even need to be busy. It might allocate several `byte[]` arrays to hold the next few incoming network requests and would typically try to avoid getting into a state where it didn’t have at least one such buffer available. The OS would have pointers to all of these and may well have supplied the network card with the corresponding physical address so that it can get to work the moment data starts to arrive. So even an idle server has certain buffers that cannot be relocated.

An alternative would be for the CLR to provide a separate nonmoving heap for these sorts of operations. Perhaps we could allocate a fixed block of memory for an I/O operation, and then copy the results into the `byte[]` array on the GC heap once the I/O has finished. But that’s also not a brilliant solution. Copying data is expensive—the more copies you make of incoming or outgoing data, the slower your server will run, so you really want network and disk hardware to copy the data directly to or from its natural location. And if this hypothetical fixed heap were more than an implementation detail of the CLR—if it were available for application code to use directly to minimize copying—that might open the door to all the memory management bugs that GC is supposed to banish.

So the CLR uses a third approach: it selectively prevents heap block relocations. The GC is free to run while I/O operations are in progress, but certain heap blocks can be *pinned*. Pinning a block sets a flag that tells the GC that the block cannot currently be moved. So, if the GC encounters such a block, it will simply leave it where it is but will attempt to relocate everything around it.

There are five ways C# code normally causes heap blocks to be pinned. You can do so explicitly using the `fixed` keyword. This allows you to obtain a raw pointer to a storage location, such as a field or an array element, and the compiler will generate code that ensures that for as long as a fixed pointer is in scope, the heap block to which it refers will be pinned. A more common way to pin a block is through interop (i.e., calls

into unmanaged code, such as an OS API). If you make an interop call to an API that requires a pointer to something, the CLR will detect when that points to a heap block, and it will automatically pin the block. By default, the CLR will unpin it automatically when the method returns. If you're calling an asynchronous API that will continue to use the memory after returning, you can use the `GCHandle` class mentioned earlier to pin a heap block until you explicitly unpin it; that's the third pinning technique.

The fourth and most common way to pin heap blocks is also the least direct: many runtime library APIs call unmanaged code on your behalf and will pin the arrays you pass in as a result. For example, the runtime libraries define a `Stream` class that represents a stream of bytes. There are several implementations of this abstract class. Some streams work entirely in memory, but some wrap I/O mechanisms, providing access to files or to the data being sent or received through a network socket. The abstract `Stream` base class defines methods for reading and writing data via `byte[]` arrays, and the I/O-based stream implementations will often pin the heap blocks containing those arrays for as long as necessary.

The fifth way is to use the `GC` class's `AllocateArray<T>` method. Instead of writing, say, `new byte[4096]`, you can write `GC.AllocateArray<byte>(4096, pinned: true)`. By passing `true` as that second argument, you are telling the CLR that you want this array to be pinned permanently. The CLR maintains an additional heap especially for this purpose called the *pinned object heap* (POH). As with the LOH, arrays in the POH will not be moved around, avoiding the overhead that pinning can otherwise cause. (The POH and corresponding `AllocateArray<T>` method do not exist on .NET Framework.)

If you are writing an application that does a lot of pinning (e.g., a lot of network I/O), you may need to think carefully about how you allocate the arrays that get pinned. Pinning does the most harm for objects allocated recently in the normal way (i.e., not using `AllocateArray<T>`) because these live in the area of the heap where most compaction activity occurs. Pinning recently allocated blocks tends to cause the ephemeral section of the heap to fragment. Memory that would normally have been recovered almost instantly must now wait for blocks to become unpinned, so by the

time the collector can get to those blocks, a lot more other blocks will have been allocated after them, meaning that a lot more work is required to recover the memory.

If pinning is causing your application problems, there will be a few common symptoms. The percentage of CPU time spent in the GC will be relatively high—anything over 10% is considered to be bad. But that alone does not necessarily implicate pinning—it could be the result of middle-aged objects causing too many full collections. So you can monitor the number of pinned blocks on the heap⁸ to see if these are the specific culprit. If it looks like excessive pinning is causing you pain, you can use `GC.AllocateArray<T>` to allocate the relevant blocks on the POH.

NOTE

Arrays allocated on the POH can be used exactly like any other kind of array, and are freed using the normal GC mechanisms. You don't need to do anything special in your code to work with arrays allocated in this way. The only difference is that their location is fixed.

The `Span<T>` and `Memory<T>` types discussed in [Chapter 18](#) can sometimes provide an alternative way to avoid pinning problems. They make it much easier than it used to be to work with memory that does not live on the GC heap. So you could sidestep pinning entirely, although you'd be taking on the responsibility for managing the relevant memory.

.NET Framework has no POH, but there's still a way to minimize the impact of pinning: try to ensure that pinning mostly happens only to objects in generation 2. If you allocate a pool of buffers and reuse them for the duration of the application, this will mean that you're pinning blocks that the GC is fairly unlikely to want to move, keeping the ephemeral generations free to be compacted at any time. The earlier you allocate the buffers, the better, because the older an object is, the less likely the GC is to want to move it, so if you're going to use this approach, you should do it during your application startup if possible.

Forcing Garbage Collections

The `System.GC` class provides a `Collect` method that allows you to force a GC to occur. You can pass a number indicating the generation you would like to collect, and the overload that takes no arguments performs a full collection. You will rarely have good reason to call `GC.Collect`. I'm mentioning it here because it comes up a lot on the web, which could easily make it seem more useful than it is.

Forcing a GC can cause problems. The GC monitors its own performance and tunes its behavior in response to your application's allocation patterns. For this to work, it needs to allow enough time between collections to get an accurate picture of how well its current settings are working. If you force collections to occur too often, it will not be able to tune itself, and the outcome will be twofold: the GC will run more often than necessary, and when it does run, its behavior will be suboptimal. Both problems are likely to increase the amount of CPU time spent in the GC.

So when would you force a collection? If you happen to know that your application has just finished some work and is about to go idle, it might be worth considering forcing a collection. GCs are usually triggered by activity, so if you know that your application is about to go to sleep—perhaps it's a service that has just finished running a batch job and will not do any more work for another few hours—you know that it won't be allocating new objects and will therefore not trigger the GC automatically. So forcing a GC would provide an opportunity to return memory to the OS before the application goes to sleep. That said, if this is your scenario, it might be worth looking at mechanisms that would enable your process to exit entirely—there are various ways in which jobs or services that are only required from time to time can be unloaded completely when they are inactive. But if that technique is inapplicable for some reason—perhaps your process has high startup costs or needs to stay running to receive incoming network requests—a forced full collection might be the next best option.

It's worth being aware that there is one way that a GC can be triggered without your application needing to do anything. When the system is running low on memory, Windows broadcasts a message to all running pro-

cesses. The CLR handles this message and forces a GC when it occurs. So even if your application does not proactively attempt to return memory, memory might be reclaimed eventually if something else in the system needs it. (This is a Windows-only feature.)

Destructors and Finalization

The CLR works hard on our behalf to find out when our objects are no longer in use. It's possible to get it to notify you of this—instead of simply removing unreachable objects, the CLR can first tell an object that it is about to be removed. The CLR calls this finalization, but C# presents it through a special syntax: to exploit finalization, you must write a destructor.

WARNING

If your background is in C++, do not be fooled by the name, or the similar syntax. As you will see, a C# destructor is different from a C++ destructor in some important ways.

[Example 7-10](#) shows a destructor. This code compiles into an override of a method called `Finalize`, which as [Chapter 6](#) mentioned, is a special method defined by the `object` base class. Finalizers are always required to call the base implementation of `Finalize` that they override. C# generates that call for us to prevent us from violating the rule, which is why it doesn't let us simply write a `Finalize` method directly. You cannot write code that invokes a finalizer—they are called by the CLR, so we do not specify an accessibility level for the destructor.

Example 7-10. Class with destructor

```
public class LetMeKnowMineEnd
{
    ~LetMeKnowMineEnd()
    {
        Console.WriteLine("Goodbye, cruel world");
    }
}
```

```
}  
}
```

The CLR does not guarantee to run finalizers on any particular schedule. First of all, it needs to detect that the object has become unreachable, which won't happen until the GC runs. If your program is idle, that might not happen for a long time; the GC normally runs only when your program is doing something, or when system-wide memory pressure causes the GC to spring into life. It's entirely possible that minutes, hours, or even days could pass between your object becoming unreachable and the CLR noticing that it has become unreachable.

Even when the CLR does detect unreachability, it still doesn't guarantee to call the finalizer straightaway. Finalizers run on a dedicated thread. Because current versions of the CLR have only one finalization thread (regardless of which GC mode you choose), a slow finalizer will cause other finalizers to wait.

In most cases, the CLR doesn't even guarantee to run finalizers at all. When a process exits, if the finalization thread hasn't already managed to run all extant finalizers, it will exit without waiting for them all to finish.

In summary, finalizers can be delayed indefinitely if your program is either idle or busy, and are not guaranteed to run. But it gets worse—you can't actually do much that is useful in a finalizer.

You might think that a finalizer would be a good place to ensure that certain work is properly completed. For example, if your object writes data to a file but buffers that data so as to be able to write a small number of large chunks rather than writing in tiny dribs and drabs (because large writes are often more efficient), you might think that finalization is the obvious place to ensure that data in your buffers has been safely flushed out to disk. But think again.

During finalization, an object cannot trust the other objects it has references to. If your object's destructor runs, your object must have become unreachable. This means it's highly likely that any other objects yours refers to have also become unreachable. The CLR is likely to discover the unreachability of groups of related objects simultaneously—if your object

created three or four objects to help it do its job, the whole lot will become unreachable at the same time. The CLR makes no guarantees about the order in which it runs finalizers. This means it's entirely possible that by the time your destructor runs, all the objects you were using have already been finalized. So, if they also perform any last-minute cleanup, it's too late to use them. For example, the `FileStream` class, which derives from `Stream` and provides access to a file, closes its file handle in its destructor. Thus, if you were hoping to flush your data out to the `FileStream`, it's too late—the file stream may well already be closed.

NOTE

To be fair, things are marginally less bad than I've made them sound so far. Although the CLR does not guarantee to run most finalizers, it will usually run them in practice. The absence of guarantees matters only in relatively extreme situations. Even so, this doesn't mitigate the fact that you cannot, in general, rely on other objects in your destructor.

Since destructors seem to be of remarkably little use—that is, you can have no idea if or when they will run, and you can't use other objects inside a destructor—then what are they for?

The main reason finalization exists at all is to make it possible to write .NET types that are wrappers for the sorts of entities that are traditionally represented by handles—things like files and sockets. These are created and managed outside of the CLR—files and sockets require the operating system to allocate resources; libraries may also provide handle-based APIs, and they will typically allocate memory on their own private heaps to store information about whatever the handle represents. The CLR cannot see these activities—all it sees is a .NET object with a field containing an integer, and it has no idea that the integer is a handle for some resource outside of the CLR. So it doesn't know that it's important that the handle be closed when the object falls out of use. This is where finalizers come in: they are a place to put code that tells something external to the CLR that the entity represented by the handle is no longer in use. The inability to use other objects is not a problem in this scenario.

NOTE

If you are writing code that wraps a handle, you should normally use one of the built-in classes that derive from `SafeHandle` or, if absolutely necessary, derive your own. This base class extends the basic finalization mechanism with some handle-oriented helpers. Furthermore, it gets special handling from the interop layer to avoid premature freeing of resources.

There are some other uses for finalization, although the unpredictability and unreliability already discussed mean there are limits to what it can do for you. Some classes contain a finalizer that does nothing other than check that the object was not abandoned in a state where it had unfinished work. For example, if you had written a class that buffers data before writing it to a file, as described previously, you would need to define some method that callers should use when they are done with your object (perhaps called `Flush` or `Close`), and you could write a finalizer that checks to see if the object was put into a safe state before being abandoned, raising an error if not. This would provide a way to discover when programs have forgotten to clean things up correctly.

If you write a finalizer, you should disable it when your object is in a state where it no longer requires finalization, because finalization has its costs. If you offer a `Close` or `Flush` method, finalization is unnecessary once these have been called, so you should call the `System.GC` class's `SuppressFinalize` method to let the GC know that your object no longer needs to be finalized. If your object's state subsequently changes, you can call the `ReRegisterForFinalize` method to reenable it.

The greatest cost of finalization is that it guarantees that your object will survive at least into the first generation and possibly beyond. Remember, all objects that survive from generation 0 make it into generation 1. If your object has a finalizer, and you have not disabled it by calling `SuppressFinalize`, the CLR cannot get rid of your object until it has run its finalizer. And since finalizers run asynchronously on a separate thread, the object has to remain alive even though it has been found to be unreachable. So the object is not yet collectable, even though it is unreachable. It therefore lives on into generation 1. It will usually be finalized shortly afterward, meaning that the object will then become a waste

of space until a generation 1 collection occurs. Those happen rather less frequently than generation 0 collections. A finalized object therefore makes inefficient use of memory, which is a reason to avoid finalization, and a reason to disable it whenever possible in objects that do sometimes require it.

WARNING

Even though `SuppressFinalize` can save you from the most egregious costs of finalization, an object that uses this technique still has higher overheads than an object with no finalizer at all. The CLR does some extra work when constructing finalizable objects to keep track of those that have not yet been finalized. (Calling `SuppressFinalize` just takes your object back out of this tracking list.) So, although suppressing finalization is much better than letting it occur, it's better still if you don't ask for it in the first place.

A slightly weird upshot of finalization is that an object that the GC discovered was unreachable can make itself reachable again. It's possible to write a destructor that stores the `this` reference in a root reference, or perhaps in a collection that is reachable via a root reference. Nothing stops you from doing this, and the object will continue to work (although its finalizer will not run a second time if the object becomes unreachable again), but it's an odd thing to do. This is referred to as *resurrection*, and just because you can do it doesn't mean you should. It is best avoided.

I hope that by now, I have convinced you that destructors do not provide a general-purpose mechanism for shutting down objects cleanly. They are mostly useful only for dealing with handles for things that live outside of the CLR's control, and it's best to avoid relying on them. If you need timely, reliable cleanup of resources, there's a better mechanism.

IDisposable

The runtime libraries define an interface called `IDisposable`. The CLR does not treat this interface as being in any way special, but C# has some built-in support for it. `IDisposable` is a simple abstraction; as [Example 7-11](#) shows, it defines just one member, the `Dispose` method.

Example 7-11. The `IDisposable` interface

```
public interface IDisposable
{
    void Dispose();
}
```

The idea behind `IDisposable` is straightforward. If your code creates an object that implements this interface, you should call `Dispose` once you’ve finished using that object (with the occasional exception—see [“Optional Disposal”](#)). This then provides the object with an opportunity to free up resources it may have allocated. If the object being disposed of was using resources represented by handles, it will typically close those handles immediately rather than waiting for finalization to kick in (and it should suppress finalization at the same time). If the object was using services on some remote machine in a stateful way—perhaps holding a connection open to a server to be able to make requests—it would immediately let the remote system know that it no longer requires the services, in whatever way is necessary (for example, by closing the connection).

NOTE

There is a persistent myth that calling `Dispose` causes the GC to do something. You may read on the web that `Dispose` finalizes the object, or even that it causes the object to be garbage collected. This is nonsense. The CLR does not handle `IDisposable` or `Dispose` differently than any other interface or method.

`IDisposable` is important because it’s possible for an object to consume very little memory and yet tie up some expensive resources. For example, consider an object that represents a connection to a database. Such an object might not need many fields—it could even have just a single field containing a handle representing the connection. From the CLR’s point of view, this is a pretty cheap object, and we could allocate hundreds of them without triggering a GC. But in the database server, things would look different—it might need to allocate a considerable amount of memory for each incoming connection. Connections might even be strictly lim-

ited by licensing terms. (This illustrates that “resource” is a fairly broad concept—it means pretty much anything that you might run out of.)

Relying on GC to notice when database connection objects are no longer in use is likely to be a bad strategy. The CLR will know that we’ve allocated, say, 50 of the things, but if that consumes only a few hundred bytes in total, it will see no reason to run the GC. And yet our application may be about to grind to a halt—if we have only 50 connection licenses for the database, the next attempt to create a connection will fail. And even if there’s no licensing limitation, we could still be making highly inefficient use of database resources by opening far more connections than we need.

It’s imperative that we close connection objects as soon as we can, without waiting for the GC to tell us which ones are out of use. This is where `IDisposable` comes in. It’s not just for database connections, of course. It’s critically important for any object that is a front for something that lives outside the CLR, such as a file or a network connection. Even for resources that aren’t especially constrained, `IDisposable` provides a way to tell objects when we’re finished with them so that they can shut down cleanly, solving the problem described earlier for objects that perform internal buffering.

If a resource is expensive to create, you may want to reuse it. This is often the case with database connections, so the usual practice is to maintain a pool of connections. Instead of closing a connection when you’re finished with it, you return it to the pool, making it available for reuse. (Many of .NET’s data access providers can do this for you.) The `IDisposable` model is still useful here. When you ask a resource pool for a resource, it usually provides a wrapper around the real resource, and when you dispose that wrapper, it returns the resource to the pool instead of freeing it. So calling `Dispose` is really just a way of saying, “I’m done with this object,” and it’s up to the `IDisposable` implementation to decide what to do next with the resource it represents.

Implementations of `IDisposable` are required to tolerate multiple calls to `Dispose`. Although this means consumers can call `Dispose` multiple times without harm, they should not attempt to use an object after it has been disposed. In fact, the runtime libraries define a special exception

that objects can throw if you misuse them in this way:

`ObjectDisposedException`. (I will discuss exceptions in [Chapter 8](#).)

You're free to call `Dispose` directly, of course, but C# also supports `IDisposable` in three ways: `foreach` loops, `using` statements, and `using` declarations. A `using` statement is a way to ensure that you reliably dispose an object that implements `IDisposable` once you're done with it. [Example 7-12](#) shows how to use it.

Example 7-12. A `using` statement

```
using (StreamReader reader = File.OpenText(@"C:\temp\File.txt"))
{
    Console.WriteLine(reader.ReadToEnd());
}
```

This is equivalent to the code in [Example 7-13](#). The `try` and `finally` keywords are part of C#'s exception handling system, which I'll discuss in detail in [Chapter 8](#). In this case, they're being used to ensure that the call to `Dispose` inside the `finally` block executes even if something goes wrong in the code inside the `try` block. This also ensures that `Dispose` gets called if you execute a `return` statement in the middle of the block. (It even works if you use a `goto` statement to jump out of it.)

Example 7-13. How `using` statements expand

```
{
    StreamReader reader = File.OpenText(@"C:\temp\File.txt");
    try
    {
        Console.WriteLine(reader.ReadToEnd());
    }
    finally
    {
        if (reader != null)
        {
            ((IDisposable) reader).Dispose();
        }
    }
}
```

If the variable type of the declaration in the `using` statement is a value type, C# will not generate the code that checks for `null` and will just invoke `Dispose` directly.

C# also offers a simpler alternative, a `using` declaration, shown in [Example 7-14](#). The difference is that we don't need to provide a block. A `using` declaration disposes its variable when the variable goes out of scope. It still generates `try` and `finally` blocks, so in cases where a `using` statement's block happens to finish at the end of some other block (e.g., it finishes at the end of a method), you can change to a `using` declaration with no change of behavior. This reduces the number of nested blocks, which can make your code easier to read. (On the other hand, with an ordinary `using` block, it may be easier to see exactly when the object is no longer used. So each style has its pros and cons.)

Example 7-14. A `using` declaration

```
using StreamReader reader = File.OpenText(@"C:\temp\File.txt");  
Console.WriteLine(reader.ReadToEnd());
```

If you need to use multiple disposable resources within the same scope, and you want to use a `using` statement, not a declaration (e.g., because you want to dispose the resources at the earliest opportunity instead of waiting for the relevant variables to go out of scope), you can nest them, but it might be easier to read if you stack multiple `using` statements in front of a single block. [Example 7-15](#) uses this to copy the contents of one file to another.

Example 7-15. Stacking `using` statements

```
using (Stream source = File.OpenRead(@"C:\temp\File.txt"))  
using (Stream copy = File.Create(@"C:\temp\Copy.txt"))  
{  
    source.CopyTo(copy);  
}
```

Stacking `using` statements is not a special syntax; it's just an upshot of the fact that a `using` statement is always followed by a single embedded

statement, which will be executed before `Dispose` gets called. Normally, that statement is a block, but in [Example 7-15](#), the first `using` statement's embedded statement is the second `using` statement. If you use `using` declarations instead, stacking is unnecessary because these don't have an associated embedded statement.

A `foreach` loop generates code that will use `IDisposable` if the enumerator implements it. [Example 7-16](#) shows a `foreach` loop that uses just such an enumerator.

Example 7-16. A `foreach` loop

```
foreach (string file in Directory.EnumerateFiles(@"C:\temp"))
{
    Console.WriteLine(file);
}
```

The `Directory` class's `EnumerateFiles` method returns an `IEnumerable<string>`. As you saw in [Chapter 5](#), this has a `GetEnumerator` method that returns an `IEnumerator<string>`, an interface that inherits from `IDisposable`. Consequently, the C# compiler will produce code equivalent to [Example 7-17](#).

Example 7-17. How `foreach` loops expand

```
{
    IEnumerator<string> e =
        Directory.EnumerateFiles(@"C:\temp").GetEnumerator();
    try
    {
        while (e.MoveNext())
        {
            string file = e.Current;
            Console.WriteLine(file);
        }
    }
    finally
    {
        if (e != null)
        {

```

```

        ((IDisposable) e).Dispose();
    }
}
}

```

There are several variations the compiler can produce, depending on the collection's enumerator type. If it's a value type that implements `IDisposable`, the compiler won't generate the check for `null` in the `finally` block (just as in a `using` statement). If the static type of the enumerator does not implement `IDisposable`, the outcome depends on whether the type is open for inheritance. If it is sealed, or if it is a value type, the compiler will not generate code that attempts to call `Dispose` at all. If it is not sealed, the compiler generates code in the `finally` block that tests at runtime whether the enumerator implements `IDisposable`, calling `Dispose` if it does and doing nothing otherwise.

The `IDisposable` interface is easiest to consume when you obtain a resource and finish using it in the same method, because you can write a `using` statement (or where applicable, a `foreach` loop) to ensure that you call `Dispose`. But sometimes, you will write a class that creates a disposable object and puts a reference to it in a field, because it will need to use that object over a longer timescale. For example, you might write a logging class, and if a logger object writes data to a file, it might hold on to a `StreamWriter` object. C# provides no automatic help here, so it's up to you to ensure that any contained objects get disposed. You would write your own implementation of `IDisposable` that disposes the other objects, as [Example 7-18](#) does. Note that this example sets `_file` to `null`, so it will not attempt to dispose the file twice. This is not strictly necessary, because the `StreamWriter` will tolerate multiple calls to `Dispose`. But it does give the `Logger` object an easy way to know that it is in a disposed state, so if we were to add some real methods, we could check `_file` and throw an `ObjectDisposedException` if it is `null`.

Example 7-18. Disposing a contained instance

```

public sealed class Logger(string filePath) : IDisposable
{
    private StreamWriter? _file = File.CreateText(filePath);

```

```

    public void Dispose()
    {
        if (_file != null)
        {
            _file.Dispose();
            _file = null;
        }
    }
    // A real class would go on to do something with the StreamWriter, of course
}

```

This example dodges an important problem. The class is sealed, which avoids the issue of how to cope with inheritance. If you write an unsealed class that implements `IDisposable`, you should provide a way for a derived class to add its own disposal logic. The most straightforward solution would be to make `Dispose` virtual so that a derived class can override it, performing its own cleanup in addition to calling your base implementation. However, there is a more complicated pattern that you will see from time to time in .NET.

Some objects implement `IDisposable` and also have a finalizer. Since the introduction of `SafeHandle` and related classes, it's relatively unusual for a class to need to provide both (unless it derives from `SafeHandle`). Only wrappers for handles normally need finalization, and classes that use handles now typically defer to a `SafeHandle` to provide that, rather than implementing their own finalizers. However, there are exceptions, and some library types implement a pattern designed to support both finalization and `IDisposable`, allowing you to provide custom behaviors for both in derived classes. For example, the `Stream` base class works this way.

WARNING

This pattern is called the *dispose pattern*, but do not take that to mean that you should normally use this when implementing `IDisposable`. On the contrary, it is extremely unusual to need this pattern. Even back when it was invented, few classes needed it, and now that we have `SafeHandle`, it is almost never necessary. (`SafeHandle` was introduced in .NET 2.0, so it has been a very long time since the dispose pattern was broadly useful.) Unfortunately, some people misunderstood the narrow utility of this pattern, so you will find a certain amount of well-intentioned but utterly wrong advice telling you that you should use this for all `IDisposable` implementations. Ignore it. The pattern's main relevance today is that you sometimes encounter it in old types such as `Stream`.

The pattern is to define a protected overload of `Dispose` that takes a single `bool` argument. The base class calls this from its public `Dispose` method and also its destructor, passing in `true` or `false`, respectively. That way, you have to override only one method, the protected `Dispose`. It can contain logic common to both finalization and disposal, such as closing handles, but you can also perform any disposal-specific or finalization-specific logic because the argument tells you which sort of cleanup is being performed. [Example 7-19](#) shows how this might look. (This is for illustration only—the `MyCustomLibraryInteropWrapper` class has been made up for this example.)

Example 7-19. Custom finalization and disposal logic

```
public class MyFunkyStream : Stream
{
    // For illustration purposes only. Usually better to avoid this whole
    // pattern and to use some type derived from SafeHandle instead.
    private IntPtr _myCustomLibraryHandle;
    private Logger? _log;

    protected override void Dispose(bool disposing)
    {
        base.Dispose(disposing);

        if (_myCustomLibraryHandle != IntPtr.Zero)
        {
            MyCustomLibraryInteropWrapper.Close(_myCustomLibraryHandle);
        }
    }
}
```

```

        _myCustomLibraryHandle = IntPtr.Zero;
    }
    if (disposing)
    {
        if (_log != null)
        {
            _log.Dispose();
            _log = null;
        }
    }
}

// ...overloads of Stream's abstract methods would go here
}

```

This hypothetical example is a custom implementation of the `Stream` abstraction that uses some external non-.NET library that provides handle-based access to resources. We prefer to close the handle when the public `Dispose` method is called, but if that hasn't happened by the time our finalizer runs, we want to close the handle then. So the code checks to see if the handle is still open and closes it if necessary, and it does this whether the call to the `Dispose(bool)` overload happened as a result of the object being explicitly disposed or being finalized—we need to ensure that the handle is closed in either case. However, this class also appears to use an instance of the `Logger` class from [Example 7-18](#). Because that's an ordinary object, we shouldn't attempt to use it during finalization, so we attempt to dispose it only if our object is being disposed. If we are being finalized, then although `Logger` itself is not finalizable, it uses a `FileStream`, which is finalizable; and it's quite possible that the `FileStream` finalizer will already have run by the time our `MyFunkyStream` class's finalizer runs, so it would be a bad idea to call methods on the `Logger`.

When a base class provides this virtual protected form of `Dispose`, it should call `GC.SuppressFinalization` in its public `Dispose`. The `Stream` base class does this. More generally, if you find yourself writing a class that offers both `Dispose` and a finalizer, then whether or not you choose to support inheritance with this pattern, you should in any case suppress finalization when `Dispose` is called.

Since I've recommended avoiding this pattern, what should code like [Example 7-18](#) do if using `sealed` is unacceptable? The answer is straightforward: if you are writing a class that implements `IDisposable` and you want that class to be open for inheritance (i.e., not `sealed`), make your `Dispose` method `virtual`. That way, derived types can override it to add their own disposal logic (and these overrides should always call the base class's `Dispose`).

Optional Disposal

Although you should call `Dispose` at some point on most objects that implement `IDisposable`, there are a few exceptions. For example, the Reactive Extensions for .NET (described in [Chapter 11](#)) provide `IDisposable` objects that represent subscriptions to streams of events. You can call `Dispose` to unsubscribe, but some event sources come to a natural end, automatically shutting down any subscriptions. If that happens, you are not required to call `Dispose`. Also, the `Task` type, which is used extensively in conjunction with the asynchronous programming techniques described in [Chapter 17](#), implements `IDisposable` but does not need to be disposed unless you cause it to allocate a `WaitHandle`, something that will not occur in normal usage. The way `Task` is generally used makes it particularly awkward to find a good time to call `Dispose` on it, so it's fortunate that it's not normally necessary.

The `HttpClient` class is another exception to the normal rules but in a different way. We rarely call `Dispose` on instances of this type, but in this case it's because we are encouraged to reuse instances. If you construct, use, and dispose an `HttpClient` each time you need one, you will defeat its ability to reuse existing connections when making multiple requests to the same server. This can cause two problems. First, opening an HTTP connection can sometimes take longer than sending the request and receiving the response, so preventing `HttpClient` from reusing connections to send multiple requests over time can cause significant performance problems. Connection reuse only works if you reuse the `HttpClient`.⁹ Second, the TCP protocol (which underpins HTTP requests unless you're using HTTP/3) has characteristics that mean the OS cannot always instantly reclaim all the resources associated with a connection: it

may need to keep the connection's TCP port reserved for a considerable time (maybe a few minutes) after you've told the OS to close the connection, and it's possible to run out of ports, preventing all further communication.

Such exceptions are unusual. It is only safe to omit calls to `Dispose` when the documentation for the class you're using explicitly states that it is not required.

Boxing

While I'm discussing GC and object lifetime, there's one more topic I should talk about in this chapter: *boxing*. Boxing is the process that enables a variable of type `object` to refer to a value type. An `object` variable is capable only of holding a reference to something on the heap, so how can it refer to an `int`? What happens when the code in [Example 7-20](#) runs?

Example 7-20. Using an `int` as an object

```
static void Show(object o)
{
    Console.WriteLine(o.ToString());
}

int num = 42;
Show(num);
```

The `Show` method expects an object, and I'm passing it `num`, which is a local variable of the value type `int`. In these circumstances, C# generates a box, which is essentially a reference type wrapper for a value. The CLR can automatically provide a box for any value type, although if it didn't, you could write your own class that does something similar. [Example 7-21](#) shows a hand-built box.

Example 7-21. Not actually how a box works

```
// Not a real box but similar in effect.
public class Box<T>(T v)
    where T : struct
{
    public readonly T Value = v;

    public override string? ToString() => Value.ToString();
    public override bool Equals(object? obj) => Value.Equals(obj);
    public override int GetHashCode() => Value.GetHashCode();
}
```

This is a fairly ordinary class that contains a single instance of a value type as its only field. If you invoke the standard members of `object` on the box, this class's overrides make it look as though you invoked them directly on the field itself. So, if I passed `new Box<int>(num)` as the argument to `Show` in [Example 7-20](#), `Show` would receive a reference to that box. When `Show` called `ToString`, the box would call the `int` field's `ToString`, so you'd expect the program to display 42.

We don't need to write [Example 7-21](#), because the CLR will build the box for us. It will create an object on the heap that contains a copy of the boxed value and forward the standard `object` methods to the boxed value. And it does some things that we can't. If you ask a boxed `int` its type by calling `GetType`, it will return the same `Type` object as you'd get if you called `GetType` directly on an `int` variable—I can't do that with my custom `Box<T>`, because `GetType` is not virtual. Also, getting back the underlying value is easier than it would be with a hand-built box, because unboxing is an intrinsic CLR feature.

If you have a reference of type `object`, and you cast it to `int`, the CLR checks to see if the reference does indeed refer to a boxed `int`; if it does, the CLR returns a copy of the boxed value. (If not, it throws an `InvalidCastException`.) So, inside the `Show` method of [Example 7-20](#), I could write `(int) o` to get back a copy of the original value, whereas if I were using the class in [Example 7-21](#), I'd need the more convoluted `((Box<int>) o).Value`.

I can also use pattern matching to extract a boxed value. [Example 7-22](#) uses a declaration pattern to detect whether the variable `o` contains a reference to a boxed `int`, and if it does, it extracts that into the local variable `i`. As we saw in [Chapter 2](#), when you use a pattern with the `is` operator like this, the resulting expression evaluates to `true` if the pattern matches and `false` if it does not. So the body of this `if` statement runs only if there was an `int` value there to be unboxed.

Example 7-22. Unboxing a value with a type pattern

```
if (o is int i)
{
    Console.WriteLine(i * 2);
}
```

Boxes are automatically available for all structs,¹⁰ not just the built-in value types. If the struct implements any interfaces, the box will provide all the same interfaces. (That's another trick that [Example 7-21](#) cannot perform.)

Some implicit conversions cause boxing. You can see this in [Example 7-20](#). I have passed an expression of type `int` where `object` was required, without needing an explicit cast. Implicit conversions also exist between a value and any of the interfaces that value's type implements. For example, you can assign a value of type `int` into a variable of type `Comparable<int>` (or pass it as a method argument of that type) without needing a cast. This causes a box to be created, because variables of any interface type are like variables of type `object`, in that they can hold only a reference to an item on the garbage-collected heap.

NOTE

Implicit boxing conversions are not implicit reference conversions. This means that they do not come into play with covariance or contravariance. For example, `IEnumerable<int>` is not compatible with `IEnumerable<object>` despite the existence of an implicit conversion from `int` to `object`, because that is not an implicit reference conversion.

Implicit boxing can occasionally cause problems for one of two reasons. First, it makes it easy to generate extra work for the GC. The CLR does not attempt to cache boxes, so if you write a loop that executes 100,000 times, and that loop contains an expression that uses an implicit boxing conversion, you'll end up generating 100,000 boxes, which the GC will eventually have to clean up just like anything else on the heap. Second, each box operation (and each unbox) copies the value, which might not provide the semantics you were expecting. [Example 7-23](#) illustrates some potentially surprising behavior.

Example 7-23. Illustrating the pitfalls of mutable structs

```
static void CallDispose(IDisposable o)
{
    o.Dispose();
}

DisposableValue dv = new ();
Console.WriteLine("Passing value variable:");
CallDispose(dv);
CallDispose(dv);
CallDispose(dv);

IDisposable id = dv;
Console.WriteLine("Passing interface variable:");
CallDispose(id);
CallDispose(id);
CallDispose(id);

Console.WriteLine("Calling Dispose directly on value variable:");
dv.Dispose();
dv.Dispose();
dv.Dispose();

Console.WriteLine("Passing value variable:");
CallDispose(dv);
CallDispose(dv);
CallDispose(dv);

public struct DisposableValue : IDisposable
{
    private bool _disposedYet;
```

```

public void Dispose()
{
    if (!_disposedYet)
    {
        Console.WriteLine("Disposing for first time");
        _disposedYet = true;
    }
    else
    {
        Console.WriteLine("Was already disposed");
    }
}
}

```

The `DisposableValue` struct implements the `IDisposable` interface we saw earlier. It keeps track of whether it has been disposed already. The program contains a `CallDispose` method that calls `Dispose` on any `IDisposable` instance. The program declares a single variable of type `DisposableValue` and passes this to `CallDispose` three times. Here's the output from that part of the program:

```

Passing value variable:
Disposing for first time
Disposing for first time
Disposing for first time

```

On all three occasions, the struct seems to think this is the first time we've called `Dispose` on it. That's because each call to `CallDispose` created a new box—we are not really passing the `dv` variable; we are passing a newly boxed copy each time, so the `CallDispose` method is working on a different instance of the struct each time. This is consistent with how value types normally work—even when there's no boxing, when you pass one as an argument, you get a copy (unless you use the `ref` or `in` keywords).

The next part of the program ends up generating just a single box—it assigns the value into another local variable of type `IDisposable`. This uses the same implicit conversion as we did when passing the variable directly as an argument, so this creates yet another box, but it does so only once.

We then pass the same reference to this particular box three times over, which explains why the output from this phase of the program looks different:

```
Passing interface variable:  
Disposing for first time  
Was already disposed  
Was already disposed
```

These three calls to `CallDispose` all use the same box, which contains an instance of our struct, and so after the first call, it remembers that it has been disposed already. Next, our program calls `Dispose` directly on the local variable, producing this output:

```
Calling Dispose directly on value variable:  
Disposing for first time  
Was already disposed  
Was already disposed
```

No boxing at all is involved here, so we are modifying the state of the local variable. Someone who only glanced at the code might not have expected this output—we have already passed the `dv` variable to a method that called `Dispose` on its argument, so it might be surprising to see that it thinks it hasn't been disposed the first time around. But once you understand that `CallDispose` requires a reference and therefore cannot use a value directly, it's clear that every call to `Dispose` before this point has operated on some boxed copy, and not the local variable.

Finally, we make three more calls passing the `dv` directly to `CallDispose` again. This is exactly what we did at the start of the code, so these calls generate yet more boxed copies. But this time, we are copying a value that's already in the state of having been disposed, so we see different output:

```
Passing value variable:  
Was already disposed  
Was already disposed  
Was already disposed
```

The behavior is all straightforward when you understand what's going on, but it requires you to be mindful that you're dealing with a value type and to understand when boxing causes implicit copying. This is one of the reasons Microsoft discourages developers from writing value types that can change their state—if a value cannot change, then a boxed value of that type also cannot change. It matters less whether you're dealing with the original or a boxed copy, so there's less scope for confusion, although it is still useful to understand when boxing will occur to avoid performance penalties.

Boxing used to be a much more common occurrence in early versions of .NET. Before generics arrived in .NET 2.0, collection classes all worked in terms of `object`, so if you wanted a resizable list of integers, you'd end up with a box for each `int` in the list. Generic collection classes do not cause boxing—a `List<int>` is able to store unboxed values directly.

Boxing Nullable<T>

[Chapter 3](#) described the `Nullable<T>` type, a wrapper that adds null value support to any value type. Remember, C# has special syntax for this, in which you can just put a question mark on the end of a value type name, so we'd normally write `int?` instead of `Nullable<int>`. The CLR has special support for `Nullable<T>` when it comes to boxing.

`Nullable<T>` itself is a value type, so if you attempt to get a reference to it, the compiler will generate code that attempts to box it, as it would with any other value type. However, at runtime, the CLR will not produce a box containing a copy of the `Nullable<T>` itself. Instead, it checks to see if the value is in a null state (i.e., its `HasValue` property returns `false`), and if so, it just returns `null`. Otherwise, it boxes the contained value. For example, if a `Nullable<int>` has a value, boxing it will produce a box of type `int`. This will be indistinguishable from the box you'd get if you had started with an ordinary `int` value. (One upshot of this is that the pattern matching shown in [Example 7-22](#) works whether the type of variable originally boxed was an `int` or an `int?`. You use `int` in the declaration pattern in either case.)

You can unbox a boxed `int` into variables of either type `int?` or `int`. So all three unboxing operations in [Example 7-24](#) will succeed. They would also succeed if the first line were modified to initialize the `boxed` variable from a `Nullable<int>` that was not in the null state. (If you were to initialize `boxed` from a `Nullable<int>` in the null state, that would have the same effect as initializing it to `null`, in which case the final line of this example would throw a `NullReferenceException`.)

Example 7-24. Unboxing an `int` to nullable and non-nullable variables

```
object boxed = 42;
int? nv = boxed as int?;
int? nv2 = (int?) boxed;
int v = (int) boxed;
```

This is a runtime feature, and not simply the compiler being clever. The IL `box` instruction, which is what C# generates when it wants to box a value, detects `Nullable<T>` values; the `unbox` and `unbox.any` IL instructions are able to produce a `Nullable<T>` value from either a `null` or a reference to a boxed value of the underlying type. So, if you wrote your own wrapper type that looked like `Nullable<T>`, it would not behave in the same way; if you assigned a value of your type into an `object`, it would box your whole wrapper just like any other value. It's only because the CLR knows about `Nullable<T>` that it behaves differently.

Summary

In this chapter, I described the heap that the runtime provides. I showed the strategy that the CLR uses to determine which heap objects can still be reached by your code, and the generation-based mechanism it uses to reclaim the memory occupied by objects that are no longer in use. The GC is not clairvoyant, so if your program keeps an object reachable, the GC has to assume that you might use that object in the future. This means you will sometimes need to be careful to make sure you don't cause memory leaks by accidentally keeping hold of objects for too long. We looked at the finalization mechanism, and its various limitations and performance

issues, and we also looked at `IDisposable`, which is the preferred system for cleaning up nonmemory resources. Finally, we saw how value types can act like reference types thanks to boxing.

In the next chapter, I will show how C# presents the error-handling mechanisms of the CLR.

- 1** The acronym GC is used throughout this chapter to refer to both the *garbage collector* mechanism and also *garbage collection*, which is what the garbage collector does.
- 2** The Mono runtime's GC shares no code with the .NET GC, even though both now live in the same GitHub repository. Nonetheless, they both use the same approach here.
- 3** Value types defined with `ref struct` are an exception: they always live on the stack. [Chapter 18](#) discusses these.
- 4** The CLR doesn't always wait until it runs out of memory. I will discuss the details later. For now, the important point is that from time to time, it will try to free up some space.
- 5** The Mono runtime uses a slightly simpler scheme, but it still relies on the basic principle of treating new and old objects differently.
- 6** .NET provides an application configuration setting that lets you change this threshold.
- 7** Rare though single-core CPUs are these days, it's still common to run in virtual machines that present only one core to the code they host. This is often the case if your application runs on a cloud-hosted service using a consumption-based tariff, for example.
- 8** Microsoft supplies various free tools that can explore GC activity including `dotnet-trace`, `dotnet-counters`, and PerfView.
- 9** Strictly speaking, it's the underlying `MessageHandler` that needs to be reused. If you obtain an `HttpClient` from an `IHttpClientFactory`, you normally dispose it because the factory holds on to the handler and reuses it across `HttpClient` instances.

10 Except for `ref struct` types, because those invariably live on the stack.